

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY**FACULTY OF TECHNOLOGY & ENGINEERING****SMT. KUNDANBEN DINSHA PATEL DEPARTMENT OF INFORMATION
TECHNOLOGY**

Subject Name: Artificial Intelligence
Subject Code: IT473

Semester: B.Tech VII
Academic Year: 2019-20

Lab Manual

General Instruction:

Please find below instruction for preparing your IT473 AI practical file.

You all have to submit ipynb file with description practical wise (AIM, Brief description of the practical, description of the dataset used (if applicable), code, intermediate description of code whenever required along with output, conclusion, question answers with supporting output and description).

PRACTICAL: 1**AIM:**

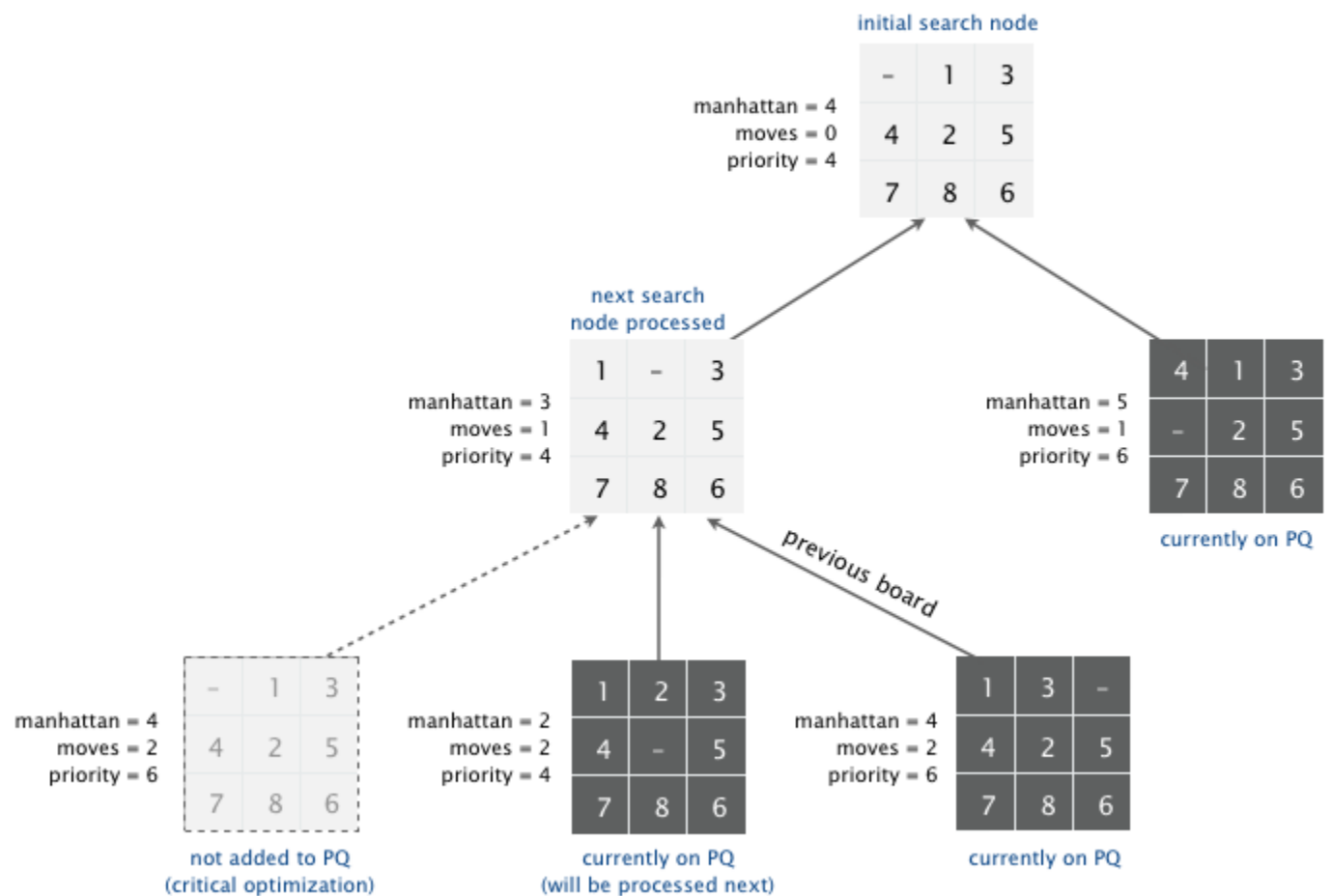
The 8 puzzle consists of eight numbered, movable tiles set in a 3x3 frame. One cell of the frame is always empty thus making it possible to move an adjacent numbered tile into the empty cell. Apply intelligence to solve given problem and Implement it.

Software Required: Jupyter Notebook, IDLE, Any Python Editor

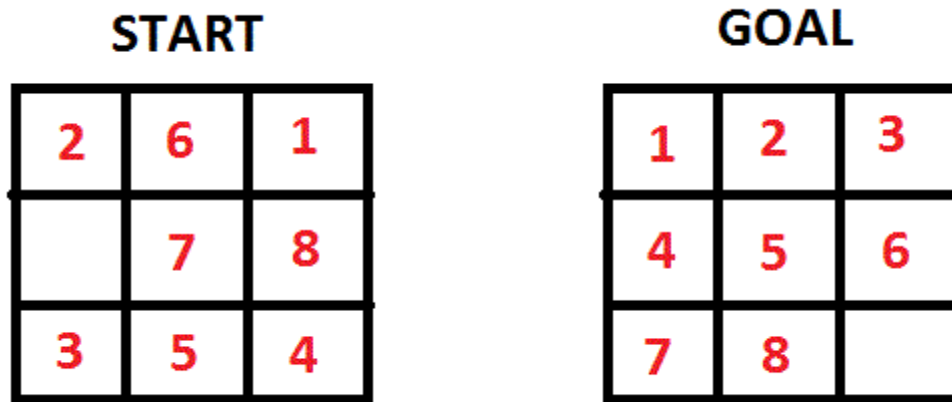
Knowledge Required: Problem Solving, Transition Operators, Heuristic Function

Theory/Logic:

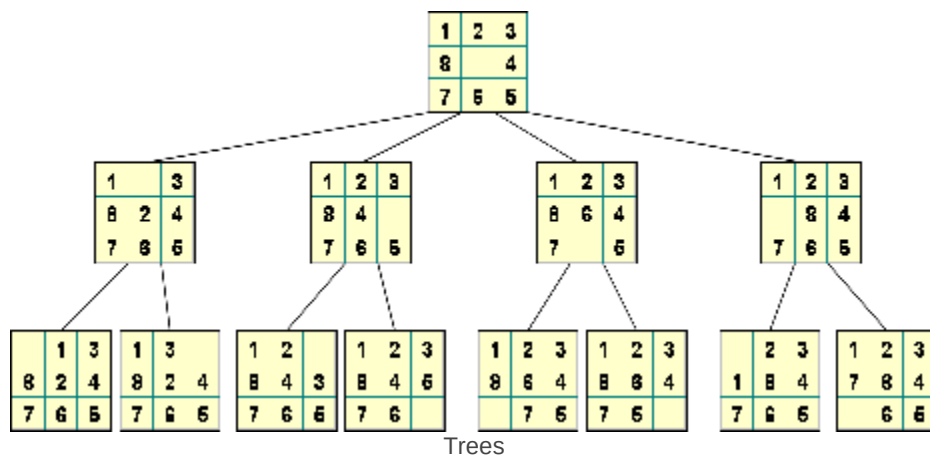
Manhattan distance the sum of vertical and horizontal distances from each tile to their goal position the puzzle also exists in other sizes particularly smaller 8 if size is 3x3 tiles called or 9 we will implement the 8 puzzle with one variation instead of having tiles numbered the puzzle also exists in other sizes particularly smaller 8 if size is 3x3 tiles called or 9.

**Goal****Initial**

8 Puzzle Programming Ignment



Lab 5 Trees Intuitions



- Admissible Heuristics

An admissible heuristic never overestimates the cost of reaching the goal.
Using an admissible heuristic will always result in an optimal solution.

- Non-Admissible Heuristics

A non-admissible heuristic may overestimate the cost of reaching the goal. It may or may not result in an optimal solution. However, the advantage is that sometimes, a non-admissible heuristic expands much fewer nodes. Thus, the total cost (= search cost + path cost) may actually be lower than an optimal solution using an admissible heuristic.

PRACTICAL: 2

AIM:

Implement Uninformed Search Strategies: BFS, DFS, and ID (Iterative Deepening-Idea of Game Searching (Chess, Checkers, Tic-Tac-Toe, Puzzle Game etc.))

Software Required: Jupyter Notebook, IDLE, Any Python Editor

Knowledge Required: Problem Solving, Transition Operators, Heuristic Function

Theory/Logic:

- **Uninformed search algorithms** or Brute-force algorithms, search through the search space all possible candidates for the solution checking whether each candidate satisfies the problem's statement.
- **Informed search algorithms** use heuristic functions that are specific to the problem, apply them to guide the search through the search space to try to reduce the amount of time spent in searching.

Breadth-first search

A Search strategy, in which the highest layer of a decision tree is searched completely before proceeding to the next layer is called Breadth-first search (BFS).

- In this strategy, no viable solutions are omitted and therefore it is guaranteed that an optimal solution is found.
- This strategy is often not feasible when the search space is large.

Algorithm

1. Create a variable called LIST and set it to be the starting state.
2. Loop until a goal state is found or LIST is empty, Do
 - a. Remove the first element from the LIST and call it E. If the LIST is empty, quit.
 - b. For every path each rule can match the state E, Do
 - (i) Apply the rule to generate a new state.
 - (ii) If the new state is a goal state, quit and return this state.
 - (iii) Otherwise, add the new state to the end of LIST.

Advantages

1. Guaranteed to find an optimal solution (in terms of shortest number of steps to reach the goal).

2. Can always find a goal node if one exists (complete).

Disadvantages

High storage requirement: exponential with tree depth.

Depth-first search

A search strategy that extends the current path as far as possible before backtracking to the last choice point and trying the next alternative path is called Depth-first search (DFS).

- This strategy does not guarantee that the optimal solution has been found.
- In this strategy, search reaches a satisfactory solution more rapidly than breadth first, an advantage when the search space is large.

Algorithm

Depth-first search applies operators to each newly generated state, trying to drive directly toward the goal.

1. If the starting state is a goal state, quit and return success.
2. Otherwise, do the following until success or failure is signalled:
 - a. Generate a successor E to the starting state. If there are no more successors, then signal failure.
 - b. Call Depth-first Search with E as the starting state.
 - c. If success is returned signal success; otherwise, continue in the loop.

Advantages

1. Low storage requirement: linear with tree depth.
2. Easily programmed: function call stack does most of the work of maintaining state of the search.

Disadvantages

1. May find a sub-optimal solution (one that is deeper or more costly than the best solution).
2. Incomplete: without a depth bound, may not find a solution even if one exists.

Bounded depth-first

search Depth-first search can spend much time (perhaps infinite time) exploring a very deep path that does not contain a solution, when a shallow solution exists. An easy way to solve this problem is to put a maximum depth bound on the search. Beyond the depth bound, a failure is generated automatically without exploring any deeper.

Problems:

1. It's hard to guess how deep the solution lies.

2. If the estimated depth is too deep (even by 1) the computer time used is dramatically increased, by a factor of bextra.
3. If the estimated depth is too shallow, the search fails to find a solution; all that computer time is wasted.

Iterative Deepening

IDDFS combines depth-first search's space-efficiency and breadth-first search's fast search (for nodes closer to root).

How does IDDFS work?

IDDFS calls DFS for different depths starting from an initial value. In every call, DFS is restricted from going beyond given depth. So basically we do DFS in a BFS fashion.

Algorithm:

```
// Returns true if target is reachable from
```

```
// src within max_depth
```

```
bool IDDFS(src, target, max_depth)
  for limit from 0 to max_depth
    if DLS(src, target, limit) == true
      return true
  return false
```

```
bool DLS(src, target, limit)
  if (src == target)
    return true;
```

```
  // If reached the maximum depth,
  // stop recursing.
  if (limit <= 0)
    return false;
```

```
  foreach adjacent i of src
    if DLS(i, target, limit-1)
      return true
```

```
  return false
```

An important thing to note is, we visit top level nodes multiple times. The last (or max depth) level is visited once, second last level is visited twice, and so on. It may seem expensive, but it turns out to be not so costly, since in a tree most of the nodes are in the bottom level. So it does not matter much if the upper levels are visited multiple times

PRACTICAL: 3

AIM:

3.1 Implement Informed (heuristic) search strategy for optimal shortest path and traffic navigational system. (Hint: A* algorithm by mentioning OPEN and CLOSED List and it takes a heuristic function as Euclidian Distance Equation)

3.2 Implement Informed (heuristic) search strategy for problem reduction and for planning good algorithms. (Hint: AO*-AND/OR graph by mentioning OPEN and CLOSED List).

A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

Why A* Search Algorithm ?

Informally speaking, A* Search algorithms, unlike other traversal techniques, it has “brains”. What it means is that it is really a smart algorithm which separates it from the other conventional algorithms. This fact is cleared in detail in below sections. And it is also worth mentioning that many games and web-based maps use this algorithm to find the shortest path very efficiently (approximation).

Explanation

Consider a square grid having many obstacles and we are given a starting cell and a target cell. We want to reach the target cell (if possible) from the starting cell as quickly as possible. Here A* Search Algorithm comes to the rescue. What A* Search Algorithm does is that at each step it picks the node according to a value-‘**f**’ which is a parameter equal to the sum of two other parameters – ‘**g**’ and ‘**h**’. At each step it picks the node/cell having the lowest ‘**f**’, and process that node/cell. We define ‘**g**’ and ‘**h**’ as simply as possible below **g** = the movement cost to move from the starting point to a given square on the grid, following the path generated to get there. **h** = the estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess.

A* Search Algorithm

We create two lists – Open List and Closed List (just like Dijkstra Algorithm)

1. Initialize the open list
2. Initialize the closed list put the starting node on the open list (you can leave its f at zero)
3. while the open list is not empty
 - a) find the node with the least f on the open list, call it "q"
 - b) pop q off the open list
 - c) generate q's 8 successors and set their parents to q
 - d) for each successor
 - i) if successor is the goal, stop search $\text{successor.g} = \text{q.g} + \text{distance between successor q and successor.h} = \text{distance from goal to successor}$ (This can be done using many ways, we will discuss three heuristics- Manhattan, Diagonal and Euclidean Heuristics)
 $\text{successor.f} = \text{successor.g} + \text{successor.h}$
 - ii) if a node with the same position as successor is in the OPEN list which has a lower f than successor, skip this successor
 - iii) if a node with the same position as successor is in the CLOSED list which has a lower f than successor, skip this successor otherwise, add the node to the open listend (for loop)
 - e) push q on the closed list end (while loop)

Practical no 4: Implement AO* Algorithm in python.

When a problem can be divided into a set of sub problems, where each sub problem can be solved separately and a combination of these will be a solution, AND-OR graphs or AND - OR trees are used for representing the solution. The decomposition of the problem or problem reduction generates AND arcs. One AND arc may point to any number of successor nodes. All these must

be solved so that the arc will rise to many arcs, indicating several possible solutions. Hence the graph is known as AND - OR instead of AND. An algorithm to find a solution in an AND - OR graph must handle AND area appropriately. A* algorithm cannot search AND - OR graphs efficiently.

AO* ALGORITHM:

1. Let G consists only to the node representing the initial state call this node INTT. Compute h' (INIT).
2. Until INIT is labeled SOLVED or h_i (INIT) becomes greater than FUTILITY, repeat the following procedure.
 - (I) Trace the marked arcs from INIT and select an unbounded node NODE.
 - (II) Generate the successors of NODE . if there are no successors then assign FUTILITY as h' (NODE). This means that NODE is not solvable. If there are successors then for each one called SUCCESSOR, that is not also an ancestor of NODE do the following
 - (a) add SUCCESSOR to graph G
 - (b) if successor is not a terminal node, mark it solved and assign zero to its h' value.
 - (c) If successor is not a terminal node, compute it h' value.
 - (III) propagate the newly discovered information up the graph by doing the following . let S be a set of nodes that have been marked SOLVED. Initialize S to NODE. Until S is empty repeat the following procedure;
 - (a) select a node from S call it CURRENT and remove it from S.

- (b) compute h' of each of the arcs emerging from CURRENT , Assign minimum h' to CURRENT.
- (c) Mark the minimum cost path as the best out of CURRENT.
- (d) Mark CURRENT SOLVED if all of the nodes connected to it through the new marked are have been labeled SOLVED.
- (e) If CURRENT has been marked SOLVED or its h' has just changed, its new status must be propagate backwards up the graph . hence all the ancestors of CURRENT are added to S.

Simple AO* procedure

1. Initialise the graph to start node
2. Traverse the graph following the current path accumulating nodes that have not yet been expanded or solved
3. Pick any of these nodes and expand it and if it has no successors call this value FUTILITY otherwise calculate only f' for each of the successors.
4. If f' is 0 then mark the node as SOLVED
5. Change the value of f' for the newly created node to reflect its successors by back propagation.
6. Wherever possible use the most promising routes and if a node is marked as SOLVED then mark the parent node as SOLVED.
7. If starting node is SOLVED or value greater than FUTILITY, stop, else repeat from 2.

PRACTICAL: 5

AIM:

Getting Started With Python Logic Programming using Kanren and SymPy packages

- Compare expressions and find out unknown values.
- Form and manipulate logic expressions using symbolic and Boolean values with SymPy package.

Compare expressions and find out unknown values.

Gearing up for logic programming with Python, we will install a couple of [packages](#). Let's use pip for this.

- **Kanren:** It lets us express logic as rules and facts and simplifies making code for business logic.

```
>>> pip install kanren
```

- **SymPy:** This is a Python library for symbolic mathematics. It is nearly a full-featured Computer Algebra System.

```
>>> pip install sympy
```

Solution Link: <https://dzone.com/articles/what-impact-might-automation-have-on-the-diversity>
<https://docs.sympy.org/latest/modules/logic.html>

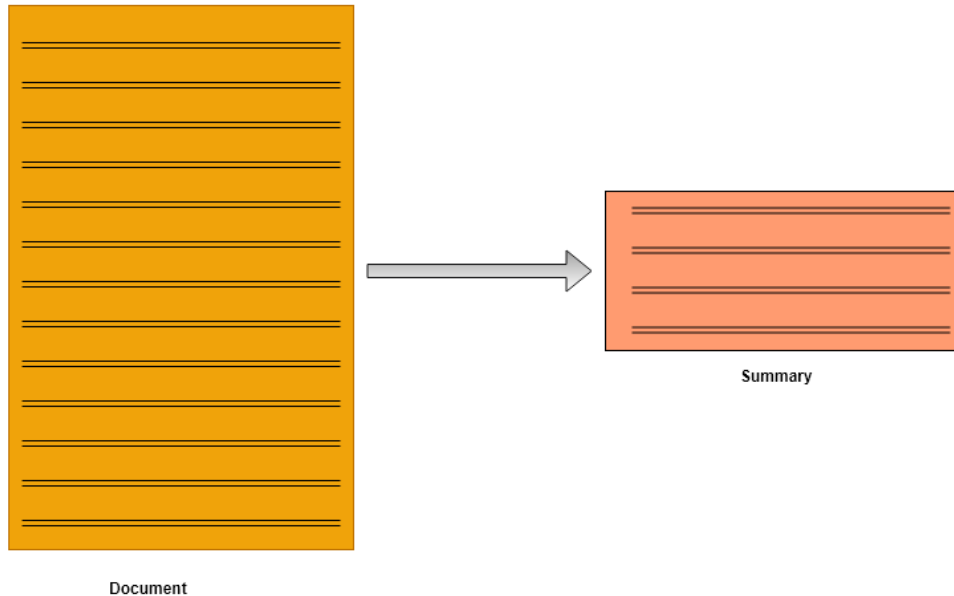
PRACTICAL: 5**AIM:**

Implement Game Search algorithm that seeks to decrease the number of nodes that are evaluated by the minimax algorithm in its search tree. It is an adversarial search algorithm used commonly for machine playing of two-player games (Tic-tac-toe, Chess, Go, etc.). It stops evaluating a move when at least one possibility has been found that proves the move to be worse than a previously examined move. Such moves need not be evaluated further.

PRACTICAL:6**AIM:**

Understand Text Summarization and Implement your own summarizer in python.

We all interact with applications which uses text summarization. Many of those applications are for the platform which publishes articles on daily news, entertainment, sports. With our busy schedule, we prefer to read the summary of those article before we decide to jump in for reading entire article. Reading a summary help us to identify the interest area, gives a brief context of the story.



Summarization can be defined as a task of producing a concise and fluent summary while preserving key information and overall meaning.

Impact

Summarization systems often have additional evidence they can utilize in order to specify the most important topics of document(s). For example, when summarizing blogs, there are discussions or comments coming after the blog post that are good sources of information to determine which parts of the blog are critical and interesting.

In scientific paper summarization, there is a considerable amount of information such as cited papers and conference information which can be leveraged to identify important sentences in the original paper.

How text summarization works

In general there are two types of summarization, **abstractive** and **extractive** summarization.

1. Abstractive Summarization: Abstractive methods select words based on semantic understanding, even those words did not appear in the source documents. It aims at producing important material in a new way. They interpret and examine the text using advanced natural language techniques in order to generate a new shorter text that conveys the most critical information from the original text.

It can be correlated to the way human reads a text article or blog post and then summarizes in their own word.

Input document → understand context → semantics → create own summary.

2. Extractive Summarization: Extractive methods attempt to summarize articles by selecting a subset of words that retain the most important points.

This approach weights the important part of sentences and uses the same to form the summary. Different algorithm and techniques are used to define weights for the sentences and further rank them based on importance and similarity among each other.

Input document → sentences similarity → weight sentences → select sentences with higher rank.

The limited study is available for abstractive summarization as it requires a deeper understanding of the text as compared to the extractive approach.

Purely extractive summaries often times give better results compared to automatic abstractive summaries. This is because of the fact that abstractive summarization methods cope with problems such as semantic representation,

inference and natural language generation which is relatively harder than data-driven approaches such as sentence extraction.

There are many techniques available to generate extractive summarization. To keep it simple, I will be using an **unsupervised learning** approach to find the sentences similarity and rank them. One benefit of this will be, you don't need to train and build a model prior start using it for your project.

It's good to understand Cosine similarity to make the best use of code you are going to see. Cosine similarity is a measure of similarity between two non-zero vectors of an inner product space that measures the cosine of the angle between them. Since we will be representing our sentences as the bunch of vectors, we can use it to find the similarity among sentences. Its measures cosine of the angle between vectors. Angle will be **0** if sentences are similar.

Next, Below is our code flow to generate summarize text: -

Input article → split into sentences → remove stop words → build a similarity matrix → generate rank based on matrix → pick top N sentences for summary.

Let's create these methods.

Import all necessary libraries

```
from nltk.corpus import stopwords
from nltk.cluster.util import cosine_distance
import numpy as np
import networkx as nx
```

Generate clean sentences

```
def read_article(file_name):
    file = open(file_name, "r")
    filedata = file.readlines()
    article = filedata[0].split(". ")
    sentences = []
    for sentence in article:
        print(sentence)
        sentences.append(sentence.replace("[^a-zA-Z]", " ").split(" "))
    sentences.pop()

    return sentences
```

Similarity matrix

This is where we will be using cosine similarity to find similarity between sentences.

```
def build_similarity_matrix(sentences, stop_words):
    # Create an empty similarity matrix
    similarity_matrix = np.zeros((len(sentences),
    len(sentences)))

    for idx1 in range(len(sentences)):
        for idx2 in range(len(sentences)):
```

```

        if idx1 == idx2: #ignore if both are same sentences
            continue
        similarity_matrix[idx1][idx2] =
sentence_similarity(sentences[idx1], sentences[idx2],
stop_words)
return similarity_matrix

```

Generate Summary Method

Method will keep calling all other helper function to keep our summarization pipeline going. Make sure to take a look at all # Steps in below code.

```

def generate_summary(file_name, top_n=5):
    stop_words = stopwords.words('english')
    summarize_text = []
    # Step 1 - Read text and tokenize
    sentences = read_article(file_name)
    # Step 2 - Generate Similary Martix across sentences
    sentence_similarity_matrix =
build_similarity_matrix(sentences, stop_words)
    # Step 3 - Rank sentences in similarity matrix
    sentence_similarity_graph =
nx.from_numpy_array(sentence_similarity_matrix)
    scores = nx.pagerank(sentence_similarity_graph)
    # Step 4 - Sort the rank and pick top sentences
    ranked_sentence = sorted(((scores[i],s) for i,s in
enumerate(sentences)), reverse=True)
    print("Indexes of top ranked_sentence order are ",
ranked_sentence)
    for i in range(top_n):
        summarize_text.append(" ".join(ranked_sentence[i][1]))
    # Step 5 - Offcourse, output the summarize text
    print("Summarize Text: \n", " ".join(summarize_text))

```


Input:

In an attempt to build an AI-ready workforce, Microsoft announced Intelligent Cloud Hub which has been launched to empower the next generation of students with AI-ready skills. Envisioned as a three-year collaborative program, Intelligent Cloud Hub will support around 100 institutions with AI infrastructure, course content and curriculum, developer support, development tools and give students access to cloud and AI services. As part of the program, the Redmond giant which wants to expand its reach and is planning to build a strong developer ecosystem in India with the program will set up the core AI infrastructure and IoT Hub for the selected campuses. The company will provide AI development tools and Azure AI services such as Microsoft Cognitive Services, Bot Services and Azure Machine Learning. According to Manish Prakash, Country General Manager-PS, Health and Education, Microsoft India, said, "With AI being the defining technology of our time, it is transforming lives and industry and the jobs of tomorrow will require a different skillset. This will require more collaborations and training and working with AI. That's why it has become more critical than ever for educational institutions to integrate new cloud and AI technologies. The program is an attempt to ramp up the institutional set-up and build capabilities among the educators to educate the workforce of tomorrow." The program aims to build up the cognitive skills and in-depth understanding of developing intelligent cloud connected solutions for applications across industry. Earlier in April this year, the company announced Microsoft Professional Program In AI as a learning track open to the public. The program was developed to provide job ready skills to programmers who wanted to hone their skills in AI and data science with a series of online courses which featured hands-on labs and expert instructors as well. This program also included developer-focused AI school that provided a bunch of assets to help build AI skills.

Output:

Envisioned as a three-year collaborative program, Intelligent Cloud Hub will support around 100 institutions with AI infrastructure, course content and curriculum, developer support, development tools and give students access to cloud and AI services. The company will provide AI development tools and Azure

AI services such as Microsoft Cognitive Services, Bot Services and Azure Machine Learning. According to Manish Prakash, Country General Manager-PS, Health and Education, Microsoft India, said, "With AI being the defining technology of our time, it is transforming lives and industry and the jobs of tomorrow will require a different skillset.

As you can see, it does a pretty good job. You can further customize it to reduce to number to character instead of lines.

It is important to understand that we have used **textrank** as an approach to rank the sentences. TextRank *does not rely on any previous training data* and can work with any arbitrary piece of text. TextRank is a general purpose *graph-based ranking* algorithm for NLP.

There are much-advanced techniques available for text summarization. If you are new to it, you can start with an interesting research paper named [Text Summarization Techniques: A Brief Survey](#)

[Survey of the State of the Art in Natural Language Generation: Core tasks, applications and evaluation](#) is a much more detailed research paper which you can go through for better understanding.

PRACTICAL:7**AIM:**

Implement Bayes rule with probabilistic inference for reasoning under uncertainty.

Bayes' theorem:

Bayes' theorem is also known as **Bayes' rule**, **Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.

In probability theory, it relates the conditional probability and marginal probabilities of two random events.

Bayes' theorem was named after the British mathematician **Thomas Bayes**. The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.

It is a way to calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$.

Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.

Example: If cancer corresponds to one's age then by using Bayes' theorem, we can determine the probability of cancer more accurately with the help of age.

Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:

As from product rule we can write:

$$1. \quad P(A \wedge B) = P(A|B) P(B) \text{ or}$$

Similarly, the probability of event B with known event A:

$$1. \quad P(A \wedge B) = P(B|A) P(A)$$

Equating right hand side of both the equations, we will get:

The above equation (a) is called as **Bayes' rule** or **Bayes' theorem**. This equation is basic of most modern AI systems for **probabilistic inference**.

It shows the simple relationship between joint and conditional probabilities. Here,

$P(A|B)$ is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.

$P(B|A)$ is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.

$P(A)$ is called the **prior probability**, probability of hypothesis before considering the evidence

$P(B)$ is called **marginal probability**, pure probability of an evidence.

In the equation (a), in general, we can write $P(B) = P(A) * P(B|A_i)$, hence the Bayes' rule can be written as:

Where $A_1, A_2, A_3, \dots, A_n$ is a set of mutually exclusive and exhaustive events.

Applying Bayes' rule:

Bayes' rule allows us to compute the single term $P(B|A)$ in terms of $P(A|B)$, $P(B)$, and $P(A)$. This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one. Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

Example-1:

Question: what is the probability that a patient has diseases meningitis with a stiff neck?

Given Data:

A doctor is aware that disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:

- The Known probability that a patient has meningitis disease is 1/30,000.
- The Known probability that a patient has a stiff neck is 2%.

Let a be the proposition that patient has stiff neck and b be the proposition that patient has meningitis. , so we can calculate the following as:

$$P(a|b) = 0.8$$

$$P(b) = 1/30000$$

$$P(a) = .02$$

Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.

Example-2:

Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is $4/52$, then calculate posterior probability $P(\text{King}|\text{Face})$, which means the drawn face card is a king card.

Solution:

$P(\text{king})$: probability that the card is King = $4/52 = 1/13$

$P(\text{face})$: probability that a card is a face card = $3/13$

$P(\text{Face}|\text{King})$: probability of face card when we assume it is a king = 1

Putting all values in equation (i) we will get:

Application of Bayes' theorem in Artificial intelligence:

Following are some applications of Bayes' theorem:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.

PRACTICAL:8**AIM:**

Implement Deep Q- Learning (Deep Q-Network) in Reinforcement Learning (RL) using Python

Question: Compare Deep RL vs. Deep Learning

Q Learning

Let's say we know the expected reward of each action at every step. This would essentially be like a cheat sheet for the agent! Our agent will know exactly which action to perform.

It will perform the sequence of actions that will eventually generate the maximum total reward. This total reward is also called the Q-value and we will formalise our strategy as:

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$

The above equation states that the Q-value yielded from being at state s and performing action a is the immediate reward $r(s,a)$ plus the highest Q-value possible from the next state s' . Gamma here is the discount factor which controls the contribution of rewards further in the future.

$Q(s',a)$ again depends on $Q(s'',a)$ which will then have a coefficient of gamma squared. So, the Q-value depends on Q-values of future states as shown here:

$$Q(s, a) \rightarrow \gamma Q(s', a) + \gamma^2 Q(s'', a) \dots \dots \gamma^n Q(s'' \dots n, a)$$

Adjusting the value of gamma will diminish or increase the contribution of future rewards.

Since this is a recursive equation, we can start with making arbitrary assumptions for all q-values. With experience, it will converge to the optimal policy. In practical situations, this is implemented as an update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

where alpha is the learning rate or step size. This simply determines to what extent newly acquired information overrides old information.

Why 'Deep' Q-Learning?

Q-learning is a simple yet quite powerful algorithm to create a cheat sheet for our agent. This helps the agent figure out exactly which action to perform.

But what if this cheatsheet is too long? Imagine an environment with 10,000 states and 1,000 actions per state. This would create a table of 10 million cells. Things will quickly get out of control!

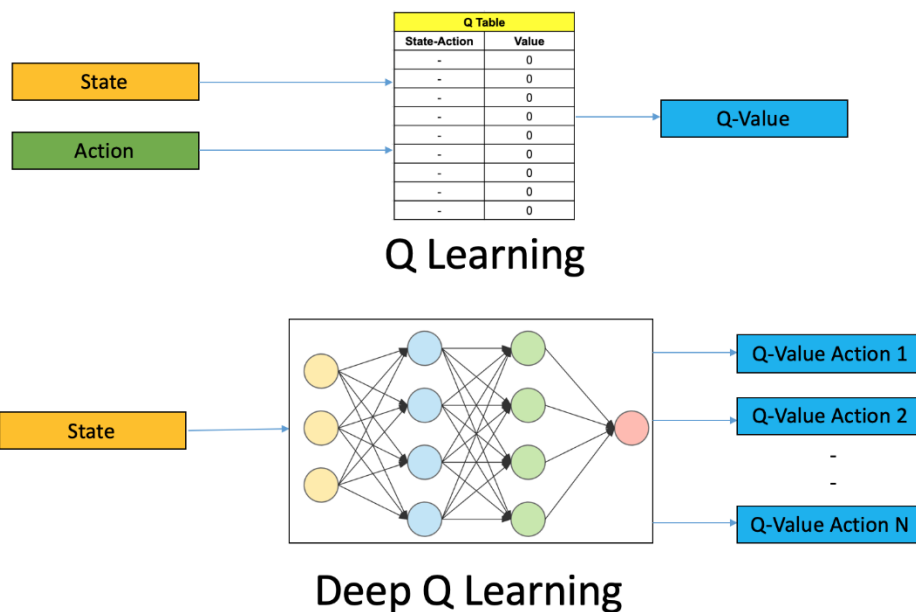
It is pretty clear that we can't infer the Q-value of new states from already explored states. This presents two problems:

- First, the amount of memory required to save and update that table would increase as the number of states increases
- Second, the amount of time required to explore each state to create the required Q-table would be unrealistic

Here's a thought – what if we approximate these Q-values with machine learning models such as a neural network? Well, this was the idea behind DeepMind's algorithm that led to its acquisition by Google for 500 million dollars!

Deep Q-Networks

In deep Q-learning, we use a neural network to approximate the Q-value function. The state is given as the input and the Q-value of all possible actions is generated as the output. The comparison between Q-learning & deep Q-learning is wonderfully illustrated below:



So, what are the steps involved in reinforcement learning using deep Q-learning networks (DQNs)?

1. All the past experience is stored by the user in memory
2. The next action is determined by the maximum output of the Q-network
3. The loss function here is mean squared error of the predicted Q-value and the target Q-value – Q^* . This is basically a regression problem. However, we do not know the target or actual value here as we are dealing with a reinforcement learning problem. Going back to the Q-value update equation derived from the Bellman equation, we have:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\boxed{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)} - Q(S_t, A_t) \right]$$

The section in green represents the target. We can argue that it is predicting its own value, but since R is the unbiased true reward, the network is going to update its gradient using backpropagation to finally converge.

Challenges in Deep RL as Compared to Deep Learning

So far, this all looks great. We understood how neural networks can help the agent learn the best actions. However, there is a challenge when we compare deep RL to deep learning (DL):

- **Non-stationary or unstable target:** Let us go back to the pseudocode for deep Q-learning:

Start with $Q_0(s, a)$ for all s, a .

Get initial state s

For $k = 1, 2, \dots$ till convergence

Sample action a , get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathbb{E}_{s' \sim P(s'|s,a)} [(Q_{\theta}(s, a) - \text{target}(s'))^2] \big|_{\theta=\theta_k}$$

$$s \leftarrow s'$$

Chasing a nonstationary target!

Updates are correlated within a trajectory!

As you can see in the above code, the target is continuously changing with each iteration. In deep learning, the target variable does not change and hence the training is stable, which is just not true for RL.

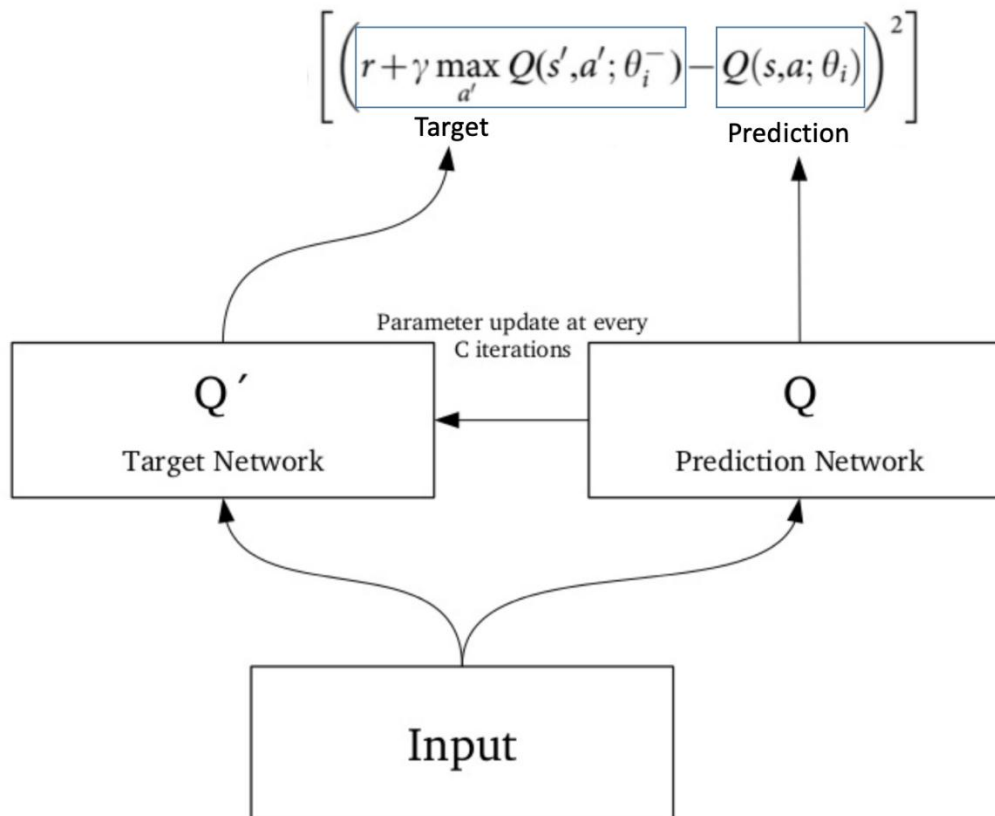
To summarise, we often depend on the policy or value functions in reinforcement learning to sample actions. However, this is frequently changing as we continuously learn what to explore. As we play out the game, we get to know more about the ground truth values of states and actions and hence, the output is also changing.

So, we try to learn to map for a constantly changing input and output. But then what is the solution?

1. Target Network

Since the same network is calculating the predicted value and the target value, there could be a lot of divergence between these two. So, instead of using 1one neural network for learning, we can use two.

We could use a separate network to estimate the target. This target network has the same architecture as the function approximator but with frozen parameters. For every C iterations (a hyperparameter), the parameters from the prediction network are copied to the target network. This leads to more stable training because it keeps the target function fixed (for a while):



2. Experience Replay

To perform experience replay, we store the agent's experiences – $et=(st,at,rt,st+1)$

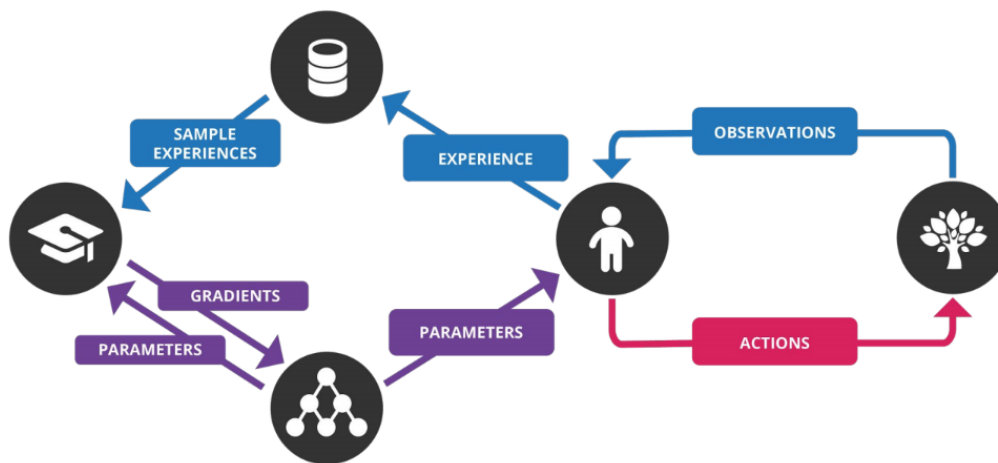
What does the above statement mean? Instead of running Q-learning on state/action pairs as they occur during simulation or the actual experience, the system stores the data discovered for [state, action, reward, next_state] – in a large table.

Let's understand this using an example.

Suppose we are trying to build a video game bot where each frame of the game represents a different state. During training, we could sample a random batch of 64 frames from the last 100,000 frames to train our network. This would get us a subset within which the correlation amongst the samples is low and will also provide better sampling efficiency.

Putting it all Together

The concepts we have learned so far? They all combine to make the deep Q-learning algorithm that was used to achieve human-level performance in Atari games (using just the video frames of the game).



I have listed the steps involved in a deep Q-network (DQN) below:

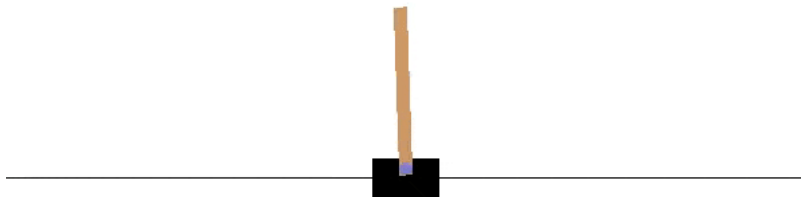
1. Preprocess and feed the game screen (state s) to our DQN, which will return the Q-values of all possible actions in the state
2. Select an action using the epsilon-greedy policy. With the probability epsilon, we select a random action a and with probability $1-\epsilon$, we select an action that has a maximum Q-value, such as $a = \text{argmax}(Q(s,a,w))$

3. Perform this action in a state s and move to a new state s' to receive a reward. This state s' is the preprocessed image of the next game screen. We store this transition in our replay buffer as $\langle s, a, r, s' \rangle$
4. Next, sample some random batches of transitions from the replay buffer and calculate the loss
5. It is known that: $Loss = (r + \gamma \max_{a'} Q(s', a'; \theta') - Q(s, a; \theta))^2$ which is just the squared difference between target Q and predicted Q
6. Perform gradient descent with respect to our actual network parameters in order to minimize this loss
7. After every C iterations, copy our actual network weights to the target network weights
8. Repeat these steps for M number of episodes

Implementing Deep Q-Learning in Python using Keras & OpenAI Gym

Alright, so we have a solid grasp on the theoretical aspects of deep Q-learning. How about seeing it in action now? That's right – let's fire up our Python notebooks!

We will make an agent that can play a game called CartPole. We can also use an Atari game but training an agent to play that takes a while (from a few hours to a day). The idea behind our approach will remain the same so you can try this on an Atari game on your machine.



CartPole is one of the simplest environments in the OpenAI gym (a game simulator). As you can see in the above animation, the goal of CartPole is to balance a pole that's connected with one joint on top of a moving cart.

Instead of pixel information, there are four kinds of information given by the state (such as the angle of the pole and position of the cart). An agent can move the cart by performing a series of actions of 0 or 1, pushing the cart left or right.

We will use the *keras-rl* library here which lets us implement deep Q-learning out of the box.

Step 1: Install keras-rl library

From the terminal, run the following code block:

```
git clone https://github.com/matthiasplappert/keras-rl.git

cd keras-rl

python setup.py install
```

Step 2: Install dependencies for the CartPole environment

Assuming you have pip installed, you need to install the following libraries:

```
pip install h5py

pip install gym
```

Step 3: Let's get started!

First, we have to import the necessary modules:

```
import numpy as np

import gym
```

```
from keras.models import Sequential

from keras.layers import Dense, Activation, Flatten

from keras.optimizers import Adam


from rl.agents.dqn import DQNAgent

from rl.policy import EpsGreedyQPolicy

from rl.memory import SequentialMemory
```

Then, set the relevant variables:

```
ENV_NAME = 'CartPole-v0'


# Get the environment and extract the number of actions available in the Cartpole problem

env = gym.make(ENV_NAME)

np.random.seed(123)

env.seed(123)

nb_actions = env.action_space.n
```

Next, we will build a very simple single hidden layer neural network model:

```
model = Sequential()

model.add(Flatten(input_shape=(1,) + env.observation_space.shape))

model.add(Dense(16))

model.add(Activation('relu'))

model.add(Dense(nb_actions))

model.add(Activation('linear'))

print(model.summary())
```

Now, configure and compile our agent. We will set our policy as Epsilon Greedy and our memory as Sequential Memory because we want to store the result of actions we performed and the rewards we get for each action.

```
policy = EpsGreedyQPolicy()

memory = SequentialMemory(limit=50000, window_length=1)

dqn = DQNAgent(model=model, nb_actions=nb_actions, memory=memory, nb_steps_warmup=10,

target_model_update=1e-2, policy=policy)

dqn.compile(Adam(lr=1e-3), metrics=['mae'])
```

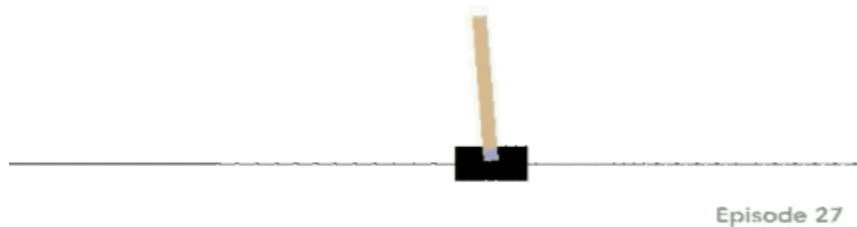
```
# Okay, now it's time to learn something! We visualize the training here for show, but this slows down training quite a lot.
```

```
dqn.fit(env, nb_steps=5000, visualize=True, verbose=2)
```

Test our reinforcement learning model:

```
dqn.test(env, nb_episodes=5, visualize=True)
```

This will be the output of our model:



Not bad! Congratulations on building your very first deep Q-learning model.

PRACTICAL:9

AIM:

Implement following Computer Vision task on Images using OpenCV.

Reading, Writing and Displaying Images	Image Filtering
Changing Color Spaces	Image Contours
Image Rotation	Scale Invariant Feature Transform (SIFT)

Image Translation	Speeded-Up Robust Features (SURF)
Image Segmentation (Watershed Algorithm)	Feature Matching
Edge Detection	Face Detection

PRACTICAL:10

AIM:

Kaggle machine learning challenges:

- Predicting Molecular Properties
- Detect objects in varied and complex images
- Detect pairs of objects in particular relationships

- Learn computer vision fundamentals with the famous MNIST data
- Predict survival on the Titanic and get familiar with ML basics
- Predict sales prices and practice feature engineering, RFs, and gradient boosting
- ImageNet Object Localization Challenge. Identify the objects in images
- Predict Future Sales

Predicting Molecular Properties

Description:

you will develop an algorithm that can predict the magnetic interaction between two atoms in a molecule (i.e., the scalar coupling constant).

Once the competition finishes, CHAMPS would like to invite the top teams to present their work, discuss the details of their models, and work with them to write a joint research publication which discusses an open-source implementation of the solution.

more details: <https://www.kaggle.com/c/champs-scalar-coupling>

Detect objects in varied and complex images:

Description:

Computer vision has advanced considerably but is still challenged in matching the precision of human perception.

Open Images is a collaborative release of ~9 million images annotated with image-level labels, object bounding boxes, object segmentation masks, and visual relationships. This uniquely large and diverse dataset is designed to spur state of the art advances in analyzing and understanding images.

This year's Open Images V5 release enabled the second Open Images Challenge to include the following 3 tracks:

Object detection track for detecting bounding boxes around object instances, relaunched from 2018.

Visual relationship detection track for detecting pairs of objects in particular relations, also relaunched from 2018.

Google AI hopes that having a single dataset with unified annotations for image classification, object detection, visual relationship detection, and instance segmentation will stimulate progress towards genuine scene understanding.

More Details: <https://www.kaggle.com/c/open-images-2019-object-detection>

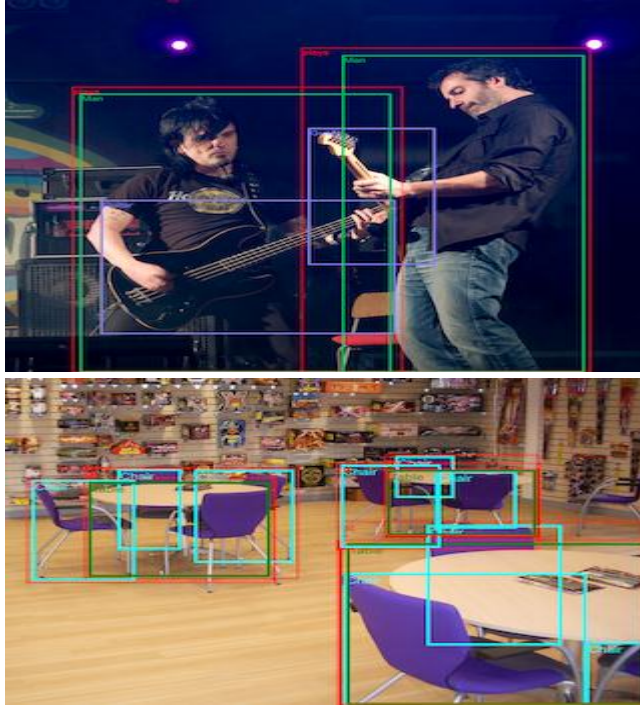
Detect pairs of objects in particular relationships:

Description:

Visual Relationship Track

In this track of the Challenge, you are asked to detect pairs of objects and the relationships that connect them.

The training set contains 329 relationship triplets with 375k training samples. These include both human-object relationships (e.g. "woman playing guitar", "man holding microphone"), object-object relationships (e.g. "beer on table", "dog inside car"), and also considers object-attribute relationships (e.g. "handbag is made of leather" and "bench is wooden").



Left: Example of 'man playing guitar' - Radiofiera - Villa Cordellina Lombardi, Montecchio Maggiore (VI) - agosto 2010 by Andrea Sartorati. Right: Example of 'chair at table' - Epic Fireworks - Loads A Room by Epic Fireworks

Learn computer vision fundamentals with the famous MNIST data:

Description:

MNIST ("Modified National Institute of Standards and Technology") is the de facto "hello world" dataset of computer vision. Since its release in 1999, this classic dataset of handwritten images has served as the basis for benchmarking classification algorithms. As new machine learning techniques emerge, MNIST remains a reliable resource for researchers and learners alike.

In this competition, your goal is to correctly identify digits from a dataset of tens of thousands of handwritten images. We've curated a set of tutorial-style kernels which cover everything from regression to neural networks. We encourage you to experiment with different algorithms to learn first-hand what works well and how techniques compare.

Practice Skills:

Computer vision fundamentals including simple neural networks

Classification methods such as SVM and K-nearest neighbors

more details: <https://www.kaggle.com/c/digit-recognizer>

Predict survival on the Titanic and get familiar with ML basics:**Description:**

The sinking of the RMS Titanic is one of the most infamous shipwrecks in history. On April 15, 1912, during her maiden voyage, the Titanic sank after colliding with an iceberg, killing 1502 out of 2224 passengers and crew. This sensational tragedy shocked the international community and led to better safety regulations for ships.

One of the reasons that the shipwreck led to such loss of life was that there were not enough lifeboats for the passengers and crew. Although there was some element of luck involved in surviving the sinking, some groups of people were more likely to survive than others, such as women, children, and the upper-class.

we ask you to complete the analysis of what sorts of people were likely to survive. In particular, we ask you to apply the tools of machine learning to predict which passengers survived the tragedy.

Practice Skills

Binary classification

Python and R basics

more details: <https://www.kaggle.com/c/titanic>

Predict sales prices and practice feature engineering, RFs, and gradient boosting:**Description:**

Competition Description



Ask a home buyer to describe their dream house, and they probably won't begin with the height of the basement ceiling or the proximity to an east-west railroad. But this playground competition's dataset proves that much more influences price negotiations than the number of bedrooms or a white-picket fence.

With 79 explanatory variables describing (almost) every aspect of residential homes in Ames, Iowa, this competition challenges you to predict the final price of each home.

Practice Skills

- Creative feature engineering
- Advanced regression techniques like random forest and gradient boosting

ImageNet Object Localization Challenge. Identify the objects in images

Description:

Competition Description

While It's pretty easy for people to identify subtle differences in photos, computers still have a ways to go. Visually similar items are tough for computers to count, like this overlapping bunch of bananas



Or consider this photo of a family of foxes camouflaged in the wild - where do the foxes end and where does the grass begins?

To solve this problem and enhance the state of the art in object detection and classification, the annual ImageNet Large Scale Visual Recognition Challenge (ILSVRC) began in 2010. This year, Kaggle is excited and honored to be the new home of the official ImageNet Object Localization competition. Participants are challenged with identifying all objects within an image, so those images can then be classified and annotated.

Already, because of this competition, there's been a $4.2\times$ reduction in image classification error (from 28.2% to 6.7%) and a $1.7\times$ reduction in localization error (from 42.5% to 25.3%) between 2010 and 2014 alone. Can you improve the accuracy even further?

more details: <https://www.kaggle.com/c/imagenet-object-localization-challenge>

Predict Future Sales:

Description:

In this competition you will work with a challenging time-series dataset consisting of daily sales data, kindly provided by one of the largest Russian software firms - [1C Company](#).

We are asking you to predict total sales for every product and store in the next month. By solving this competition you will be able to apply and enhance your data science skills.

more details: <https://www.kaggle.com/c/competitive-data-science-predict-future-sales>

PRACTICAL:11

AIM:

Case Study1: Natural language processing and Expert System.

Case Study2: Briefly mention any three artificial intelligence libraries in Python

Prepare document based on Case study.